Universidad Católica Andrés Bello

Facultad de Ingeniería

Escuela de Ingeniería Informática

Cátedra: Tópicos Especiales de Programación

Proyecto 1: Motor analítico LogiExpress en Prolog y Haskell

Docentes:

Mora, Noraimar

Bello, Jannelly

Integrantes:

José Andrés Pereira

Aracelis Valles

Michel Márquez

Puerto Ordaz, noviembre de 2026

Índice

1. Introducción 3
2. Modelado de datos 3
2.1 Modelado lógico (Prolog) 3
2.2 Modelado funcional (Haskell) 4
2.3 Comparación del modelado 4
3. Funciones y predicados investigados 4
3.1 Prolog 4
3.2 Haskell 6
4. Resumen de actividades y responsables 6
4.1 Historial de commits 7
5. Evidencias de ejecución por consola 7
5.1 Prolog 7
5.2 Haskell 11
6. Conclusiones 12

1. Introducción

La empresa de transporte aéreo LogiExpress necesita un motor analítico que determine rutas válidas entre ciudades. El motor debe respetar los días de operación de cada vuelo, el tiempo mínimo de conexión (TMC) entre escalas, la ausencia de ciclos y un límite opcional de escalas. Además, debe encontrar la ruta de menor costo y la de menor tiempo total de viaje.

El problema se modela como un grafo dirigido y ponderado: cada ciudad es un nodo y cada vuelo es una arista con hora de salida, hora de llegada, precio y días de operación. La base de conocimiento se lee de forma dinámica desde el archivo plano `vuelos.txt`, que contiene 20 vuelos entre 8 ciudades venezolanas.

El sistema se implementó dos veces, bajo dos paradigmas distintos:

- **Paradigma lógico (Prolog):** los vuelos son hechos dinámicos y las rutas se obtienen por backtracking y unificación.
- **Paradigma funcional (Haskell):** los vuelos son valores inmutables de un tipo algebraico y las rutas se generan con una búsqueda en profundidad expresada como list comprehension.

Ambas soluciones ofrecen un menú interactivo en consola. Este informe describe el modelado de datos, las funciones y predicados investigados, el reparto del trabajo entre los integrantes, las pruebas de ejecución y una comparación entre los dos paradigmas.

2. Modelado de datos

Cada línea de `vuelos.txt` tiene siete campos separados por `|`:

```
Pto Ordaz|Maiquetia|07:00|08:00|5600|Vlo#1|Diario
```

En ambos lenguajes las horas HH:MM se convierten a **minutos desde la medianoche** (07:00 = 420). Así, validar una conexión o calcular la duración de una ruta se reduce a sumas y restas de enteros.

2.1 Modelado lógico (Prolog)

Cada vuelo se guarda como un **hecho dinámico** en la base de conocimiento. El predicado `cargar_vuelos/1` lee el archivo, separa sus líneas y, por cada una, inserta un hecho con `assertz/1`:

```
:- dynamic vuelo/7.
:- dynamic tmc/1.

% vuelo(origen, destino, salida, llegada, precio, id, dias)
guardar_vuelo(Linea) :-
    split_string(Linea, "|", " ", [Org, Dest, HSal, HLleg, Pre, Id, D]),
    hora_a_min(HSal, Sal),
    hora_a_min(HLleg, Lleg),
    number_string(Precio, Pre),
    split_string(D, "-", " ", Dias),
    assertz(vuelo(Org, Dest, Sal, Lleg, Precio, Id, Dias)).
```

Por ejemplo, la primera línea de vuelos.txt queda guardada como el hecho vuelo("Pto Ordaz", "Maiquetia", 420, 480, 5600, "Vlo#1", ["Diario"]).

- Los días de operación se guardan como lista: `"L-M-V"` se convierte en `["L","M","V"]`.
- El TMC es un hecho dinámico, `tmc(45)`. El menú lo modifica con `retractall/1` y `assertz/1`, sin reiniciar el programa.
- Una ruta es una **lista de términos** `v(Org, Dest, Salida, Llegada, Precio, Id)`, construida con el patrón `[Cabeza|Cola]`.

El código está dividido en dos archivos: `main.pl` (carga, consultas básicas y menú) y `rutas.pl` (búsqueda, ordenamiento y optimización). `main.pl` carga el segundo con `ensure_loaded/1`.

2.2 Modelado funcional (Haskell)

Los vuelos se representan con un **tipo de dato algebraico** con sintaxis de registro, y el grafo es una lista de vuelos:

```
type Minutos = Int
type Grafo = [Vuelo]

data Vuelo = Vuelo
  { origen :: String, destino :: String, salida :: Minutos, llegada :: Minutos
  , precio :: Double, identificador :: String, dias :: [String]
  } deriving (Eq, Show)

newtype Ruta = Ruta { vuelos :: [Vuelo] } deriving (Eq, Show)
```

- El parseo devuelve `Either String Vuelo`: una línea mal formada produce un mensaje de error en lugar de detener el programa.
- Los datos son **inmutables**. Cambiar el TMC o recargar el archivo no modifica nada: el menú se vuelve a llamar con el nuevo valor y el nuevo grafo como argumentos.

2.3 Comparación del modelado

Aspecto	Prolog	Haskell
Vuelo	Hecho `vuelo/7` en la base de conocimiento	Valor del tipo `Vuelo`
Grafo	Conjunto implícito de hechos	Lista explícita `[Vuelo]`
Ruta	Lista de términos `v/6`	`newtype Ruta` sobre `[Vuelo]`
TMC	Hecho mutable `tmc/1`	Argumento que se pasa al menú
Errores de lectura	Fallo del predicado	Valor `Left` de `Either`

3. Funciones y predicados investigados

Esta sección recoge lo que el equipo investigó de cada lenguaje para resolver el problema.

3.1 Prolog

Predicado	Uso en el proyecto
`assertz/1`, `retractall/1`, `:- dynamic`	Cargar los vuelos en la base de conocimiento y

Predicado	Uso en el proyecto
	cambiar el TMC en tiempo de ejecución
`read_file_to_string/3`, `split_string/4`	Leer `vuelos.txt` y separar líneas, campos (`|`), horas (`:`) y días (`-`)
`number_string/2`	Convertir texto a número (horas, precios, TMC)
`findall/3`	Recolectar en una lista todas las rutas que produce el backtracking
`forall/2`	Recorrer los vuelos de una ciudad para mostrarlos
`member/2`, `\+`	Verificar días de operación y evitar ciudades ya visitadas
`last/2`	Obtener el último vuelo de una ruta para calcular su duración
`format/2`, `format/3`	Mostrar resultados y convertir minutos a HH:MM
`catch/3`, `read_line_to_string/2`	Leer datos del menú sin que una entrada inválida detenga el programa

Backtracking en la búsqueda de rutas. El predicado `buscar/7` tiene dos cláusulas: una para el vuelo que llega directamente al destino y otra para el vuelo hacia una escala, desde la cual sigue buscando de forma recursiva. Prolog prueba todos los vuelos que unifican con la ciudad actual y retrocede cuando alguna condición falla. Así recorre todo el espacio de soluciones sin bucles explícitos.

```
% vuelo a una escala y de ahi seguir buscando
buscar(Org, Dest, Dia, Hora, Visitadas, Max,
[v(Org,Esc,S,L,P,Id)|Resto]) :-
    Max > 1,
    vuelo(Org, Esc, S, L, P, Id, Dias),
    Esc \== Dest,
    \+ member(Esc, Visitadas),
    opera(Dias, Dia),
    a_tiempo(Hora, S),
    Max1 is Max - 1,
    buscar(Esc, Dest, Dia, L,
    [Esc|Visitadas], Max1, Resto).
```

En esta cláusula, \+ member(Esc, Visitadas) realiza el control de ciclos; opera(Dias, Dia) verifica la compatibilidad de frecuencia; a_tiempo(Hora, S) valida que la salida sea mayor o igual a la llegada anterior más el TMC; y Max1 is Max - 1 aplica el límite de escalas K.

Acumuladores. La lista `Visitadas` funciona como acumulador: cada llamada recursiva agrega la nueva escala, de modo que una ciudad nunca se repite en el mismo itinerario.

Mergesort sobre listas. No se usaron `sort/2` ni `msort/2`. El ordenamiento se implementó con tres predicados: `dividir/3` parte la lista en dos mitades, `ordenar/3` ordena cada mitad recursivamente y `mezclar/4` une las dos mitades ordenadas. El criterio (`costo` o `tiempo`) se pasa como argumento, de modo que el mismo algoritmo sirve para ambos reportes.

3.2 Haskell

Función o construcción	Uso en el proyecto
`data`, `type`, `newtype`, registros	Definir `Vuelo`, `Grafo`, `Minutos` y `Ruta`
`Either`, `Maybe`, `readMaybe`	Parseo seguro de horas, precios y del límite de escalas
`lines`, `words`, `unwords`	Leer el archivo por líneas y limpiar espacios
`foldr`	Implementar `splitOn` para separar los campos por `|` y los días por `-`
`map`, `filter`, `any`, `concatMap`	Consultas de salidas y llegadas, normalización de texto y verificación de días
List comprehensions	Búsqueda en profundidad (DFS) de rutas con todas las restricciones
`sortOn`, `take` (Data.List)	Ordenar rutas por costo o tiempo y tomar la mejor
Composición `(.)`	Encadenar funciones, por ejemplo `take 1 . sortOn costoRuta . rutas ...`
`try`, `IOException`	Manejar el error si `vuelos.txt` no existe

DFS con list comprehension. La función `rutasConTMC` expresa la búsqueda como una list comprehension recursiva. Cada guarda corresponde a una restricción del enunciado:

```
        [ ruta
        | vuelo <- grafo
        , normalizar (origen vuelo) == ciudad
        , operaEl dia vuelo
        , normalizar (destino vuelo) `notElem` visitadas
        , compatible camino vuelo
        , ruta <- buscar (normalizar (destino vuelo)) (vuelo : camino)
            (normalizar (destino vuelo) : visitadas)
        ]
```

En esta expresión, operaEl dia vuelo verifica el día de operación; `notElem` visitadas realiza el control de ciclos; y compatible camino vuelo valida que la salida sea mayor o igual a la llegada anterior más el TMC.

El límite de escalas se representa con `Maybe Int`, donde `Nothing` significa sin límite. Por la **evaluación perezosa**, la lista de rutas no se calcula al definirla, sino cuando el menú la necesita (al mostrarla u ordenarla con `sortOn`). Como `sortOn` tiene que comparar todas las rutas, en este caso las rutas se generan completas de todas formas.

4. Resumen de actividades y responsables

El trabajo se dividió por paradigma: un integrante desarrolló la solución en Haskell y dos integrantes se repartieron la solución en Prolog, cada uno en su propio archivo para evitar conflictos en Git.

Integrante	Módulo	Predicados o funciones implementados
José Andrés Pereira	Haskell completo y	`Vuelo`, `Grafo`, `Ruta`, `parsearVuelo`,

Integrante	Módulo	Predicados o funciones implementados
	estructura del repositorio	`cargarVuelos`, `salidasDe`, `llegadasA`, `rutasConTMC`, `optimizarCosto`, `optimizarTiempo`, menú; `vuelos.txt`, `.gitignore`, README inicial
Michel Márquez	Prolog: `main.pl` (Módulo 1 y menú)	`cargar_vuelos/1`, `guardar_vuelo/1`, `hora_a_min/2`, `hora_texto/2`, `salidas/1`, `llegadas/1`, `iniciar/0`, `menu/0`, `ejecutar/1`, cambio de TMC; sección de Prolog del README
Aracelis Valles	Prolog: `rutas.pl` (Módulos 2 y 3)	`opera/2`, `a_tiempo/2`, `ruta/5`, `buscar/7`, `costo/2`, `tiempo/2`, mergesort (`ordenar/3`, `dividir/3`, `mezclar/4`), `mejor_ruta/6`, `reporte/5`, `mostrar_rutas/2`, `mostrar_vuelos/1`, `min_a_hora/2`

4.1 Historial de commits

Fecha	Autor	Mensaje del commit
04/11/2026	Aracelis Valles	feat: calculo de costos, tiempos, mergesort, mejor ruta y reporte formateado
03/11/2026	Michel Márquez	docs: instrucciones de ejecucion de Prolog en README
02/11/2026	Michel Márquez	feat: opciones de rutas en el menu y carga del modulo rutas.pl
02/11/2026	Aracelis Valles	busqueda de rutas con dias, TMC, control de ciclos y limite de escalas
01/11/2026	Michel Márquez	feat: menu interactivo iniciar con consultas de salidas, llegadas y TMC
01/11/2026	Michel Márquez	feat: consultas de salidas y llegadas por ciudad
31/10/2026	Michel Márquez	feat: lectura de vuelos.txt y conversion de horas
24/10/2026	José Andrés Pereira	feat: README.md actualizado para el programa en Haskell
24/10/2026	José Andrés Pereira	feat: Programa en Haskell sin segmentar
24/10/2026	José Andrés Pereira	chore: gitignore para archivos generados por la compilacion del programa en Haskell
24/10/2026	José Andrés Pereira	feat: Lista de vuelos del documento PDF del proyecto
21/10/2026	José Andrés Pereira	chore: init repository

5. Evidencias de ejecución por consola

La solución en Prolog se validó con 13 casos de prueba cuyo resultado se calculó a mano a partir de `vuelos.txt`. Los 13 coincidieron con lo esperado.

5.1 Prolog

Ejecución: `swipl prolog/main.pl` y luego `iniciar.`

#	Opción	Entrada	Resultado obtenido	Requisito validado
1	Salidas	Maiquetia	5 vuelos: Vlo#2, #3, #10, #14, #17	Consulta de partidas
2	Llegadas	Porlamar	3 vuelos: Vlo#2, #9, #15	Consulta de llegadas
3	Rutas	Pto Ordaz → Porlamar, lunes, sin límite	3 rutas (10400, 11000 y 13200 Bs)	Generación de rutas válidas
4	Rutas	Igual, K = 1	2 rutas; se excluye la de 2 escalas	Límite de escalas K
5	Rutas	Igual, K = 0	"No hay rutas para ese dia"	Límite de escalas K
6	Más barata	Maiquetia → Maracaibo, lunes	8100 Bs (Vlo#10 + Vlo#11)	Costo mínimo
7	Más barata	Pto Ordaz → Porlamar, lunes	10400 Bs (Vlo#1 + Vlo#2)	Costo mínimo
8	Reporte	Pto Ordaz → Porlamar, lunes, tiempo	180, 400 y 420 min, en ese orden	Mergesort por tiempo
9	Reporte	Pto Ordaz → Porlamar, martes, costo	Solo 2 rutas; Vlo#9 no opera los martes	Días de operación
10	Cambiar TMC	70	"TMC cambiado a 70 min"	TMC configurable
11	Más barata	Pto Ordaz → Porlamar, lunes (TMC 70)	11000 Bs (Vlo#6 + Vlo#9)	Validación de escala con TMC
12	Opción 9	—	"Opcion o dato no valido"	Manejo de entradas inválidas
13	Llegadas	porlamar (minúsculas)	"No hay vuelos que lleguen a porlamar"	Ciudad inexistente

Las pruebas 10 y 11 muestran el efecto del TMC. Con 45 minutos, el Vlo#1 (llega 08:00) conecta con el Vlo#2 (sale 09:00). Con 70 minutos, los 60 minutos de espera ya no alcanzan, la ruta se descarta y la más barata pasa a costar 11000 Bs.

```
[28 ?- iniciar.
 Vuelos cargados.

 ===== LogiExpress =====
 1. Salidas de una ciudad
 2. Llegadas a una ciudad
 3. Rutas validas
 4. Ruta mas barata
 5. Ruta mas rapida
 6. Reporte ordenado
 7. Cambiar TMC
 0. Salir
 0
 Hasta luego.
 true.
```

Figura 1. Menú principal de Prolog con las 7 opciones (`iniciar.`).

```
[6 ?- llegadas("Porlamar").                                                    ]
 No hay vuelos que lleguen a Porlamar
 true.

 7 ?- cargar_vuelos("vuelos.txt").
 true.

 8 ?- salidas("Maiquetia").
 Salidas desde Maiquetia:
   Vlo#2  Maiquetia 09:00 -> Porlamar 10:00  (4800 Bs)
   Vlo#3  Maiquetia 15:00 -> Pto Ordaz 16:00  (5300 Bs)
   Vlo#10  Maiquetia 06:30 -> Barquisimeto 07:30  (4200 Bs)
   Vlo#14  Maiquetia 11:00 -> Barcelona 11:50  (4100 Bs)
   Vlo#17  Maiquetia 07:00 -> San Antonio 08:30  (8500 Bs)
 true.

[9 ?- llegadas("Porlamar").                                                    ]
 Llegadas a Porlamar:
   Vlo#2  Maiquetia 09:00 -> Porlamar 10:00  (4800 Bs)
   Vlo#9  Valencia 17:00 -> Porlamar 18:00  (4300 Bs)
   Vlo#15  Barcelona 13:00 -> Porlamar 13:40  (3500 Bs)
 true.

 10 ?-
```

Figura 2. Consultas de salidas desde Maiquetia y llegadas a Porlamar. La primera consulta, hecha antes de cargar el archivo, muestra el aviso de que no hay vuelos.

```
[26 ?- cargar_vuelos("vuelos.txt").                                            ]
 true.

 27 ?- findall(R, ruta("Pto Ordaz", "Porlamar", "L", 100, R), Rs), length(Rs, N).

 Rs = [[v("Pto Ordaz", "Maiquetia", 420, 480, 5600, "Vlo#1"), v("Maiquetia", "Por
 lamar", 540, 600, 4800, "Vlo#2")], [v("Pto Ordaz", "Maiquetia", 420, 480, 5600,
 "Vlo#1"), v("Maiquetia", "Barcelona", 660, 710, 4100, "Vlo#14"), v("Barcelona",
 "Porlamar", 780, 820, 3500, "Vlo#15")], [v("Pto Ordaz", "Valencia", 660, 750, 67
 00, "Vlo#6"), v("Valencia", "Porlamar", 1020, 1080, 4300, "Vlo#9")]],
 N = 3.
```

Figura 3. Búsqueda de rutas Pto Ordaz → Porlamar para el lunes con `findall/3`: se obtienen 3 rutas como listas de términos `v/6`.

```
 ===== LogiExpress =====
 1. Salidas de una ciudad
 2. Llegadas a una ciudad
 7. Cambiar TMC
 0. Salir
[Opcion: 7                                                                     ]
 TMC actual: 45 min
[Nuevo TMC: 60                                                                 ]
 TMC cambiado a 60 min
```

Figura 4. Cambio del TMC desde el menú (de 45 a 60 minutos).

```
Opcion: 3
Origen: Pto Ordaz
Destino: Porlamar
[Dia (L, M, Mi, J, V, S, D): L
[Max escalas (Enter = sin limite): 1
[Ruta 1: costo 10400, tiempo 180 min
[   Vlo#1: Pto Ordaz 07:00 -> Maiquetia 08:00
[   Vlo#2: Maiquetia 09:00 -> Porlamar 10:00
 Ruta 2: costo 11000, tiempo 420 min
    Vlo#6: Pto Ordaz 11:00 -> Valencia 12:30
    Vlo#9: Valencia 17:00 -> Porlamar 18:00

Opcion: 4
Origen: Maiquetia
Destino: Maracaibo
[Dia (L, M, Mi, J, V, S, D): L
[Mejor ruta por costo (8100):
[   Vlo#10: Maiquetia 06:30 -> Barquisimeto 07:30
[   Vlo#11: Barquisimeto 08:30 -> Maracaibo 09:30

Opcion: 5
Origen: Pto Ordaz
Destino: Porlamar
[Dia (L, M, Mi, J, V, S, D): L
[Mejor ruta por tiempo (180):
[   Vlo#1: Pto Ordaz 07:00 -> Maiquetia 08:00
[   Vlo#2: Maiquetia 09:00 -> Porlamar 10:00
```

Figura 5. Opciones 3, 4 y 5 del menú: rutas válidas con máximo 1 escala, ruta más barata (8100 Bs) y ruta más rápida (180 min).

```
 Opcion: 6
 Origen: Pto Ordaz
 Destino: Porlamar
[Dia (L, M, Mi, J, V, S, D): L
[Ordenar por (costo/tiempo): tiempo
[Ruta 1: costo 10400, tiempo 180 min
[   Vlo#1: Pto Ordaz 07:00 -> Maiquetia 08:00
[   Vlo#2: Maiquetia 09:00 -> Porlamar 10:00
 Ruta 2: costo 13200, tiempo 400 min
    Vlo#1: Pto Ordaz 07:00 -> Maiquetia 08:00
    Vlo#14: Maiquetia 11:00 -> Barcelona 11:50
    Vlo#15: Barcelona 13:00 -> Porlamar 13:40
 Ruta 3: costo 11000, tiempo 420 min
    Vlo#6: Pto Ordaz 11:00 -> Valencia 12:30
    Vlo#9: Valencia 17:00 -> Porlamar 18:00
```

Figura 6. Opción 6 del menú: reporte de rutas Pto Ordaz → Porlamar ordenado por tiempo con mergesort.

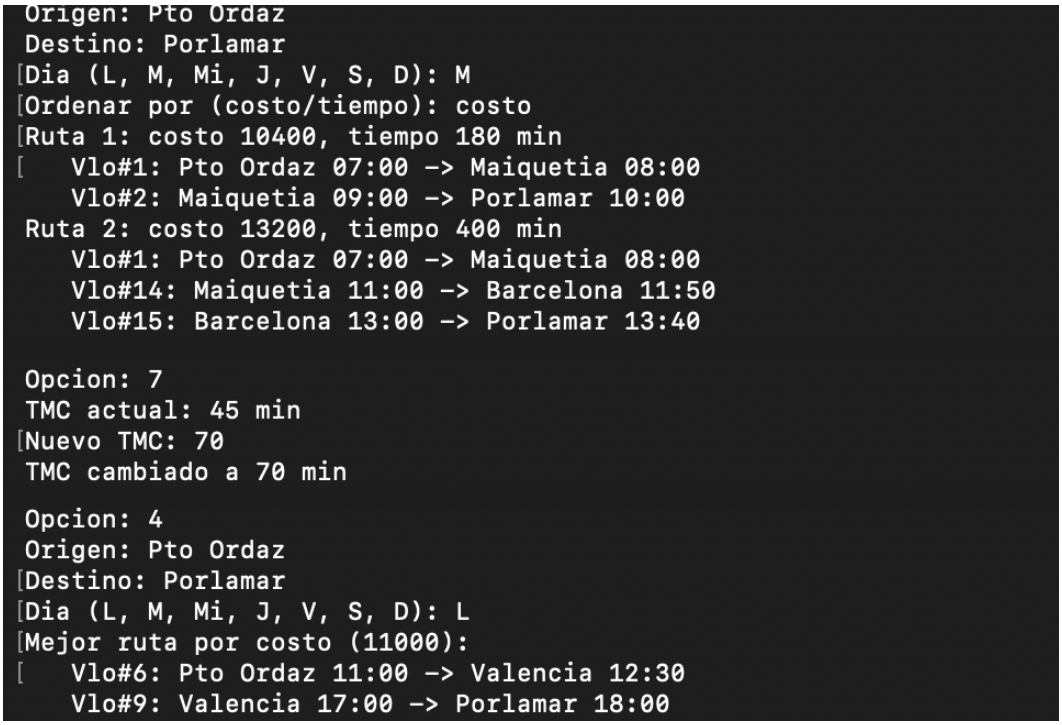


Figura 7. Reporte del martes (el Vlo#9 no opera ese día) y efecto del TMC de 70 minutos sobre la ruta más barata.

5.2 Haskell

Ejecución: `ghc haskell/main.hs -o logiexpress` y luego `./logiexpress` (en Windows, `.\logiexpress.exe`). Las pruebas se hicieron con el TMC configurado en 60 minutos.

#	Opción	Entrada	Resultado obtenido	Requisito validado
1	Cargar	—	"Vuelos cargados: 20"	Lectura de `vuelos.txt`
2	Configurar TMC	60	Nuevo TMC aplicado	TMC configurable
3	Salidas	Pto Ordaz	3 vuelos: Vlo#1, #4, #6	Consulta de partidas
4	Llegadas	Maiquetia	4 vuelos: Vlo#1, #5, #12, #20	Consulta de llegadas
5	Rutas	Barquisimeto → San Antonio, lunes	"No se encontraron rutas"	Caso sin solución
6	Menor costo	Barquisimeto → Maracaibo, lunes	Vlo#11, 3900 Bs, 60 min	Costo mínimo
7	Menor tiempo	Pto Ordaz → Maracaibo, miércoles, K = 2	Vlo#4, 9000 Bs, 120 min	Menor tiempo y límite K
8	Reporte	Maiquetia → Barcelona, por tiempo	Vlo#14, 4100 Bs, 50 min	Reporte ordenado

```
Vuelos cargados: 20

1) Cargar/recargar  2) Configurar TMC  3) Salidas  4) Llegadas
5) Rutas  6) Menor costo  7) Menor tiempo  8) Reporte  9) Salir
Opcion: 1
Vuelos cargados: 20

1) Cargar/recargar  2) Configurar TMC  3) Salidas  4) Llegadas
5) Rutas  6) Menor costo  7) Menor tiempo  8) Reporte  9) Salir
Opcion: 2
Nuevo TMC en minutos: 60

1) Cargar/recargar  2) Configurar TMC  3) Salidas  4) Llegadas
5) Rutas  6) Menor costo  7) Menor tiempo  8) Reporte  9) Salir
Opcion: 3
Ciudad: Pto Ordaz
Vlo#1: Pto Ordaz -> Maiquetia (07:00-08:00, 5600.0)
Vlo#4: Pto Ordaz -> Maracaibo (13:00-15:00, 9000.0)
Vlo#6: Pto Ordaz -> Valencia (11:00-12:30, 6700.0)

1) Cargar/recargar  2) Configurar TMC  3) Salidas  4) Llegadas
5) Rutas  6) Menor costo  7) Menor tiempo  8) Reporte  9) Salir
Opcion: 4
Ciudad: Maiquetia
Vlo#1: Pto Ordaz -> Maiquetia (07:00-08:00, 5600.0)
Vlo#5: Maracaibo -> Maiquetia (17:00-18:00, 5500.0)
Vlo#12: Barquisimeto -> Maiquetia (18:00-19:00, 4500.0)
Vlo#20: San Antonio -> Maiquetia (15:00-16:30, 8800.0)
```

Figura 8. Haskell: carga del archivo, cambio de TMC a 60 minutos y consultas de salidas y llegadas.

```
1) Cargar/recargar  2) Configurar TMC  3) Salidas  4) Llegadas
5) Rutas  6) Menor costo  7) Menor tiempo  8) Reporte  9) Salir
Opcion: 5
Origen: Barquisimeto
Destino: San Antonio
Dia: L
Maximo de escalas (vacio = sin limite):
No se encontraron rutas.
```

Figura 9. Haskell: búsqueda de rutas sin solución (Barquisimeto → San Antonio, lunes).

```
1) Cargar/recargar  2) Configurar TMC  3) Salidas  4) Llegadas
5) Rutas  6) Menor costo  7) Menor tiempo  8) Reporte  9) Salir
Opcion: 6
Origen: Barquisimeto
Destino: Maracaibo
Dia: L
Maximo de escalas (vacio = sin limite):
Barquisimeto -> Maracaibo | vuelos: Vlo#11 | escalas: 0 | costo: 3900.0 | tiempo: 60 min

1) Cargar/recargar  2) Configurar TMC  3) Salidas  4) Llegadas
5) Rutas  6) Menor costo  7) Menor tiempo  8) Reporte  9) Salir
Opcion: 7
Origen: Pto Ordaz
Destino: Maracaibo
Dia: Mi
Maximo de escalas (vacio = sin limite): 2
Pto Ordaz -> Maracaibo | vuelos: Vlo#4 | escalas: 0 | costo: 9000.0 | tiempo: 120 min

1) Cargar/recargar  2) Configurar TMC  3) Salidas  4) Llegadas
5) Rutas  6) Menor costo  7) Menor tiempo  8) Reporte  9) Salir
Opcion: 8
Origen: Maiquetia
Destino: Barcelona
Dia: Diario
Orden (costo/tiempo): tiempo
Maiquetia -> Barcelona | vuelos: Vlo#14 | escalas: 0 | costo: 4100.0 | tiempo: 50 min
```

Figura 10. Haskell: ruta de menor costo, ruta de menor tiempo con límite de escalas y reporte ordenado por tiempo.

Los resultados de ambas soluciones coinciden con los datos de `vuelos.txt`.

6. Conclusiones

Ambas soluciones cumplen con los requerimientos del enunciado y muestran las particularidades de cada paradigma: Prolog resulta más natural e intuitivo para la búsqueda con restricciones, al delegar el recorrido al backtracking automático y a la unificación del motor de inferencia,

aunque exige cautela con el estado global mutable y con la eficiencia según el orden de las metas. Haskell, en cambio, aporta robustez, inmutabilidad y seguridad mediante su sistema de tipos estricto y la evaluación perezosa, a costa de explicitar la búsqueda y de separar rigurosamente el cálculo puro de la entrada y salida.

Asimismo, la representación del tiempo en minutos desde la medianoche simplificó la validación de las escalas y el cálculo de las duraciones totales en ambos enfoques. En conclusión, Prolog prioriza la expresividad lógica directa, mientras que Haskell ofrece mayor control y prevención de errores.

Repositorio: https://github.com/Queik5450/LogiExpress-Motor-Analitico